

Day 8 + Day 9

IoT / EMBEDDED SYSTEMS

Daily Technical Report

Field	Details
Project Title	IoT Training Program
Report / Day No.	Day 8 & Day 9
Date	26/7/2026 – 27/7/2026
Prepared By	Malak Alsharqawi
Organization	Cyber Robot

Confidential — Internal Training Documentation

Table of Contents

Table of Contents.....	2
1. Daily Overview	4
1.1 Summary	4
1.2 Tasks Completed Today	4
Task 1: BME280 Environmental Monitor & Data Logger over ESP32 Access Point (AP)	4
1. Objective	4
2. Procedure	5
Step 1 — Wire the BME280 over I2C.....	5
Step 2 — Determine the correct I2C address.....	5
Step 3 — Mount flash storage for the CSV log.....	6
Step 4 — Configure the ESP32 as an Access Point (AP), not a Station (STA)	6
Step 5 — Serve the dashboard, JSON feed, and CSV export	6
Step 6 — Combine everything on a non-blocking schedule	6
3. Schematic	7
4. Photos	7
5. Network Architecture	8
6. Issues & Troubleshooting	8
7. Results & Outcome	9
2. Key Technical Concepts	9
2.1 BME280 Environmental Sensor & I2C Addressing.....	9
Technical specifications	9
Pin configuration.....	10
Measured parameters	10
Communication protocols	10
Required Arduino / PlatformIO libraries	10
Basic read loop.....	11
Understanding the SDO pin and I2C addressing	11
Why doesn't my module have an SDO pin?.....	12
Three ways to find the I2C address	12
Advantages, limitations, and key takeaways.....	12
2.2 Non-Blocking Sampling with millis()	12
The problem with delay()	12
The solution: millis()	13
Why subtraction, not addition	13
Program flow.....	13

Real-world scheduling in this project.....	13
2.3 Persistent Storage on ESP32: Flash, NVS, EEPROM Emulation, and SPIFFS/LittleFS.....	13
Flash vs. NVS: not the same thing	14
Why Preferences (NVS) replaces EEPROM.....	14
SPIFFS and LittleFS: real files, not variables.....	15
Performance characteristics	15
Wear leveling	15
Decision guidance	16
Full comparison.....	16
2.4 ESP32 Wi-Fi Modes — STA, AP, and AP+STA	16
1. Station mode (STA)	16
2. Access Point mode (AP) — used in this project	16
3. AP + STA mode	16
Important Wi-Fi terms	16
2.5 Configuring and Using Access Point (AP) Mode.....	17
Configuration steps.....	17
Built-in DHCP server.....	17
AP mode vs. STA mode	17
Advantages and limitations.....	17
Applied workflow for this project	17
2.6 HTTP Request Routing with server.on()	18
Route 1 — serving the dashboard ("/")	18
Route 2 — live JSON data ("/data")	18
Route 3 — CSV download ("/download").....	18
Execution flow summary	19
3. References.....	19

1. Daily Overview

1.1 Summary

Day 8 and Day 9 focused on turning the ESP32 into a self-contained environmental monitoring station. A BME280 sensor was interfaced over I2C to sample temperature, relative humidity, and barometric pressure, and the on-board storage options (NVS/Preferences versus SPIFFS/LittleFS) were compared to decide how sensor history should be logged. The ESP32 was then configured as its own wireless network — an Access Point (AP), not a Station (STA) joining an existing router — so that the device works as a standalone unit that any laptop or phone can connect to directly, without depending on local Wi-Fi credentials. Finally, a lightweight HTTP server was built with `server.on()` route handlers to serve a live dashboard, a JSON data feed, and a downloadable CSV log, all sampled on a non-blocking `millis()` schedule instead of `delay()`.

Beyond the working build, the two days also covered the underlying platform concepts that explain **why** each design choice was made rather than just **how** to wire it up: how the BME280's SDO address-selection pin works (and why four-pin breakout modules hide it); why `delay()` is incompatible with a device that must also serve HTTP requests; how Flash memory, NVS, EEPROM emulation, and SPIFFS/LittleFS relate to one another as software layers over the same physical chip; and how the ESP32's Wi-Fi driver differs between Station, Access Point, and combined AP+STA modes. These concepts are documented in full in Section 2, drawing on the reference notes recorded in the shared Notion notebook for this session.

1.2 Tasks Completed Today

- BME280 wiring and I2C address verification (0x76/0x77) using an I2C bus scanner, after confirming the four-pin breakout used in this project hides the SDO address-selection pin internally
- Non-blocking sensor polling and CSV logging design using `millis()` instead of `delay()`, including the overflow-safe subtraction pattern (`currentTime - lastReadTime >= interval`)
- Comparison of NVS/Preferences vs SPIFFS/LittleFS vs EEPROM emulation for persistent storage, covering how each sits on top of the same Flash chip, wear leveling, and which is appropriate for small settings versus appendable files
- Configured the ESP32 as a Wi-Fi Access Point (AP mode) using `WiFi.mode(WIFI_AP)` and `WiFi.softAP()` — deliberately not Station (STA) mode — including SSID/password rules and the built-in DHCP server behavior
- Built `server.on()` routes (`"/"`, `"/data"`, `"/download"`) to serve the dashboard, live JSON readings, and the CSV export, streaming files directly from flash instead of loading them into RAM

Task 1: BME280 Environmental Monitor & Data Logger over ESP32 Access Point (AP)

1. Objective

Interface a BME280 sensor to an ESP32 over I2C, sample temperature, humidity, and pressure on a non-blocking schedule, persist the readings to a CSV log in flash storage, and serve a live dashboard plus a CSV download — with the ESP32 broadcasting its own Wi-Fi network in Access Point (AP) mode rather than joining an existing router in Station (STA) mode, so the monitor works as a standalone device anywhere.

2. Procedure

Step 1 — Wire the BME280 over I2C

- Used the 4-pin I2C breakout (VIN, GND, SDA, SCL) — the version used in this project; the SDO address-selection pin is not exposed on this module.
- Connected VIN to 3.3V, GND to GND, SDA to GPIO 21, and SCL to GPIO 22 — the ESP32's default I2C pins.

BME280 Pin	ESP32 Pin	Purpose
VIN	3.3V	Sensor power supply
GND	GND	Common ground
SDA	GPIO 21	I2C data line (default)
SCL	GPIO 22	I2C clock line (default)

Step 2 — Determine the correct I2C address

- This module's SDO pin is tied internally by the manufacturer, so the address could not be read off the board by inspection.
- Ran an I2C bus scanner (Wire.begin(21, 22) plus a 1–127 address sweep) to confirm the sensor answers at 0x76.
- As a fallback, the firmware still tries bme.begin(0x76) then bme.begin(0x77) in case the module is swapped.

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);          // ESP32 default I2C pins
  Serial.println("Scanning I2C bus...");
}

void loop() {
  byte error, address;
  int devicesFound = 0;

  for (address = 1; address < 127; address++) {
    Wire.beginTransmission(address);
    error = Wire.endTransmission();
    if (error == 0) {
      Serial.print("I2C device found at address 0x");
      if (address < 16) Serial.print("0");
      Serial.println(address, HEX);
      devicesFound++;
    }
  }
  if (devicesFound == 0) Serial.println("No I2C devices found.");
  Serial.println("-----");
  delay(3000);
}
```

Scanner output confirming the address used in this project:

```
Scanning I2C bus...
I2C device found at address 0x76
-----
```

Step 3 — Mount flash storage for the CSV log

- Compared NVS/Preferences (key-value, wear-levelled, best for small settings) against SPIFFS/LittleFS (real files, best for appendable logs like CSV) and EEPROM emulation (legacy blob-based, not recommended for new work). Full reasoning is in Section 2.3.
- Chose LittleFS-style flash storage for the CSV file since the log needs to grow one row at a time and be downloaded as a normal file.
- Called `storage.begin(true)` so the partition is auto-formatted only the first time it fails to mount, and the CSV header row is written once on first boot.

Step 4 — Configure the ESP32 as an Access Point (AP), not a Station (STA)

- Declared a fixed SSID and password (`AP_SSID` / `AP_PASSWORD`, minimum 8 characters) and switched the radio with `WiFi.mode(WIFI_AP)`.
- Started broadcasting with `WiFi.softAP(AP_SSID, AP_PASSWORD)` so the ESP32 creates its own network instead of depending on a router.
- Retrieved the gateway address with `WiFi.softAPIP()` — the default 192.168.4.1 — since the ESP32 itself now runs the DHCP server for any client that joins.

```
#include <WiFi.h>

const char* AP_SSID      = "ESP32_Monitor";
const char* AP_PASSWORD = "12345678";

void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_AP);
  if (WiFi.softAP(AP_SSID, AP_PASSWORD)) {
    Serial.println("Access Point Started");
    Serial.print("SSID: "); Serial.println(AP_SSID);
    Serial.print("IP Address: "); Serial.println(WiFi.softAPIP());
  } else {
    Serial.println("Failed to Start Access Point");
  }
}
```

Step 5 — Serve the dashboard, JSON feed, and CSV export

- Registered `server.on("/", ...)` to stream the dashboard page from flash storage using `server.streamFile()`, which sends the file chunk-by-chunk rather than loading it into RAM.
- Registered `server.on("/data", ...)` to return the latest temperature, humidity, and pressure as a small JSON payload for the browser to poll via `fetch('/data')`.
- Registered `server.on("/download", ...)` to stream the logged CSV file back to the browser as a file attachment, using a Content-Disposition header so the browser downloads rather than renders it.

Full route implementations and a line-by-line breakdown of each handler are given in Section 2.6.

Step 6 — Combine everything on a non-blocking schedule

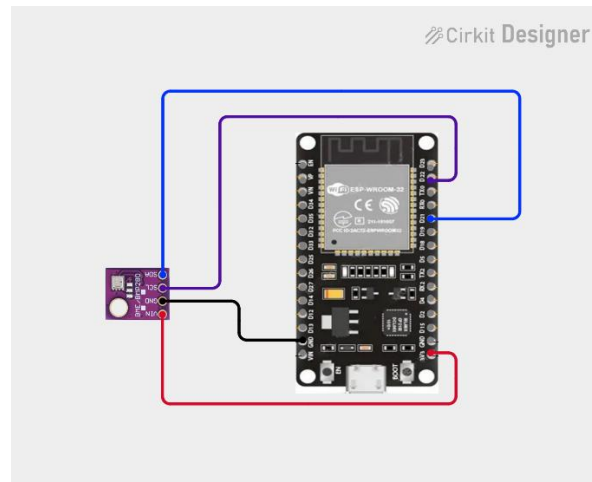
- Replaced `delay()`-based timing with `millis()` so the BME280 is polled and a new CSV row is appended roughly every 10 seconds while `server.handleClient()` keeps answering browser requests in the same `loop()` without ever blocking.

```
unsigned long lastReadTime = 0;
const unsigned long interval = 10000;    // 10 seconds

void loop() {
  unsigned long currentTime = millis();
  if (currentTime - lastReadTime >= interval) {
    lastReadTime = currentTime;
    // read BME280, append CSV row
  }
  server.handleClient();    // never blocked by the code above
}
```

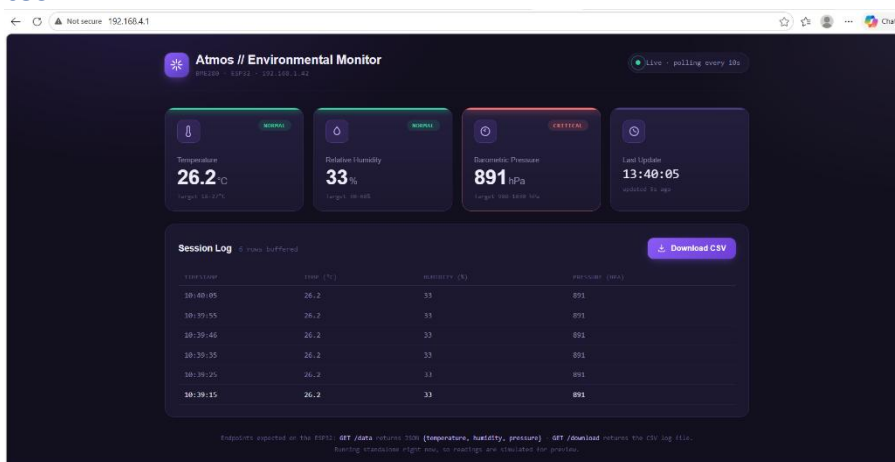
3. Schematic

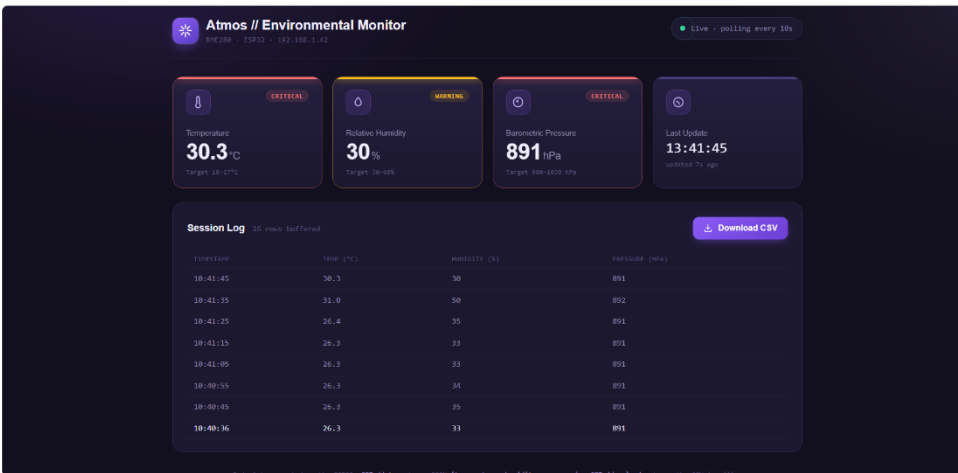
The diagram below shows the BME280 wired to the ESP32 over I2C, and the ESP32 broadcasting its own Access Point so a laptop or phone can connect directly to it.



VIN and GND supply the sensor from the ESP32's 3.3V rail; SDA/SCL on GPIO 21/22 carry the I2C bus. Once powered, the ESP32 broadcasts the ESP32_Monitor network at 192.168.4.1 instead of joining a router.

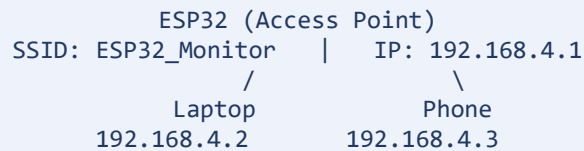
4. Photos





5. Network Architecture

Access Point (AP) mode — used in this project:



Instead of joining a router, client devices connect directly to the ESP32's own network. The ESP32 assigns each client an address itself (192.168.4.2, 192.168.4.3, ...) through its built-in DHCP server, and the dashboard is always reachable at the fixed address 192.168.4.1 — no router, internet connection, or existing Wi-Fi credentials are required. This was chosen over Station (STA) mode specifically so the monitor works as a portable, self-contained device in any location.

Typical connection sequence, from power-on to a client viewing the dashboard:

```

ESP32 boots -> WiFi.mode(WIFI_AP) -> WiFi.softAP() -> Broadcast SSID
-> Laptop finds network -> User connects -> ESP32 assigns IP
-> Client opens http://192.168.4.1
  
```

6. Issues & Troubleshooting

Symptom: Could not confirm the BME280's I2C address from the board silkscreen alone.

Root cause — the breakout used in this project only exposes four pins (VIN, GND, SDA, SCL); the SDO address-selection pin is tied internally by the manufacturer, so the resulting I2C address (0x76 or 0x77) cannot be read off the board by inspection.

Resolution: ran a standard I2C bus scanner (a 1–127 address sweep over `Wire.beginTransmission()` / `Wire.endTransmission()`) to confirm the module answers at 0x76, then hardcoded that address with a 0x77 fallback in `bme.begin()` so the sketch still recovers automatically if the module is ever replaced.

Decision point: STA mode vs AP mode for this project.

Station (STA) mode would have required the ESP32 to join an existing router and would tie the dashboard's address to whatever the local network's DHCP server assigns, meaning the device only works on networks whose credentials are already known. Since the goal for this task was a portable, standalone monitor that any laptop or phone can reach immediately — without configuring router credentials — the ESP32 was deliberately configured in Access Point (AP) mode with `WiFi.mode(WIFI_AP)` and `WiFi.softAP()`, giving a fixed, predictable address (192.168.4.1) that works the same way anywhere.

Decision point: which flash storage mechanism for the CSV log.

Preferences/NVS is optimized for small key-value settings (a Wi-Fi password, a calibration offset) and does not model an appendable file well. EEPROM emulation on the ESP32 is a single binary blob layered on top of NVS — rewriting the whole blob for every new sensor reading would be wasteful and is only recommended for porting legacy Arduino code. SPIFFS/LittleFS, by contrast, exposes true files that support opening in append mode and streaming, which matches a growing `data.csv` far more naturally. LittleFS was chosen over SPIFFS specifically because it is the actively maintained successor with better power-loss recovery.

7. Results & Outcome

The BME280 was confirmed on the I2C bus at address 0x76 and returned stable temperature, humidity, and pressure readings. Sampling and CSV logging ran on a 10-second `millis()` cycle without ever blocking the HTTP server, and the ESP32 broadcast its own Access Point (SSID: `ESP32_Monitor`) reachable at 192.168.4.1 without needing a router. The `/`, `/data`, and `/download` routes correctly served the live dashboard page, a fresh JSON reading on each poll, and the accumulated CSV log as a downloadable file that opened cleanly with a timestamp, temperature, humidity, and pressure column per row.

2. Key Technical Concepts

This section expands on the reference material recorded in the Day 8 + Day 9 Notion notebook, covering the theory behind every design decision made in Task 1.

2.1 BME280 Environmental Sensor & I2C Addressing

The BME280 is a digital environmental sensor developed by Bosch Sensortec. It measures three environmental parameters — temperature, relative humidity, and atmospheric (barometric) pressure — from a single compact module. Unlike the BMP280, which measures only temperature and pressure, the BME280 also includes a humidity sensor, making it well suited to environmental monitoring and IoT applications such as this one.

Technical specifications

Parameter	Value
Operating Voltage	1.8V – 3.6V (many breakout boards support 3.3V and 5V via onboard regulators)
Communication	I2C or SPI
Temperature Range	-40°C to +85°C
Humidity Range	0% – 100% RH
Pressure Range	300 – 1100 hPa

Parameter	Value
I2C Addresses	0x76 or 0x77
Power Consumption	Very low — suitable for battery-powered / low-power IoT applications

Pin configuration

The BME280 is commonly available as a 4-pin (I2C only) or 6-pin (I2C and SPI) breakout module. This project uses the 4-pin variant.

Pin	Description
VIN	Power supply (3.3V or 5V depending on the breakout board)
GND	Ground
SDA	I2C data line
SCL	I2C clock line

The 6-pin variant adds CS (Chip Select, SPI) and SDO (SPI MISO / I2C address selection).

Measured parameters

Temperature = 25.4 °C

Humidity = 52.8 %

Pressure = 1009.8 hPa

The sensor library returns pressure in Pa, so the firmware divides by 100 to display hPa (1 hPa = 100 Pa):

```
float pressure = bme.readPressure() / 100.0F;
```

Communication protocols

The BME280 supports two communication methods. This project uses I2C.

Protocol	Wires	Notes
I2C	SDA, SCL	Two-wire bus, easier wiring, allows multiple devices on the same bus — most common choice with ESP32/Arduino, and the method used in this project
SPI	MOSI, MISO, SCK, CS	Faster than I2C but requires more wires — typically used when higher communication speed is required

Required Arduino / PlatformIO libraries

```
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
```

```
lib_deps =
  adafruit/Adafruit Unified Sensor
```

Basic read loop

```
Adafruit_BME280 bme; // I2C
#define BME280_I2C_ADDRESS 0x76

void setup() {
  Serial.begin(115200);
  if (!bme.begin(BME280_I2C_ADDRESS)) {
    Serial.println("Could not find a valid BME280 sensor, check wiring!");
    while (1);
  }
  Serial.println("BME280 sensor initialized successfully!");
}

void loop() {
  Serial.print("Temperature: "); Serial.print(bme.readTemperature()); Serial.println("
  \xC2\xB0C");
  Serial.print("Humidity: ");    Serial.print(bme.readHumidity());    Serial.println(" %");
  Serial.print("Pressure: ");    Serial.print(bme.readPressure() / 100.0F); Serial.println("
  hPa");
  Serial.println();
  delay(2000);
}
```

Understanding the SDO pin and I2C addressing

SDO (Serial Data Out) is a pin on the BME280 whose function depends on the communication protocol: in SPI mode it acts as the MISO line; in I2C mode it is not used for data at all — instead it selects which of the two possible I2C addresses the sensor answers to.

SDO Connection	I2C Address
Connected to GND	0x76
Connected to 3.3V (VCC)	0x77

If the wrong address is used, `bme.begin()` fails and the sensor will not initialize. Not every I2C device uses an address-selection pin — some have a fixed address, some offer several pins for multiple addresses, and some are configured purely in software:

Method	Description	Example
Fixed address	Address cannot be changed	DS3231 RTC
Address selection pin	A dedicated pin selects one of two addresses	BME280 (SDO), MPU6050 (AD0), BH1750 (ADDR)
Multiple address pins	Several pins provide multiple possible addresses	PCF8574 I/O Expander
Software configurable	Address changed through software registers	Some advanced I2C peripherals

Different manufacturers name the same kind of pin differently — SDO (BME280), AD0 (MPU6050), ADDR (BH1750), SA0 (SSD1306 OLED), or A0/A1/A2 (PCF8574) — but they all serve the purpose of selecting the device's I2C address.

Why doesn't my module have an SDO pin?

Many inexpensive BME280 breakout boards expose only four pins (VIN, GND, SDA, SCL) — the version used in this project. On these modules the SDO pin is wired internally by the manufacturer, which means the address is fixed, cannot be changed without modifying the hardware, and cannot be determined simply by looking at the board.

Three ways to find the I2C address

```
if (!bme.begin(0x76)) {  
  if (!bme.begin(0x77)) {  
    Serial.println("BME280 not found!");  
    while (1);  
  }  
}
```

Advantages, limitations, and key takeaways

- Advantages: three environmental measurements in one sensor, high accuracy, factory calibrated, very low power consumption, supports both I2C and SPI, compact, and well supported by Arduino/PlatformIO libraries.
- Limitations: measures relative (not absolute) humidity; altitude estimates from pressure are approximate unless a reference sea-level pressure is supplied; requires the correct I2C address (0x76 or 0x77) when using I2C.
- Every I2C device must have a unique bus address; SDO is simply the BME280's name for its address-selection pin, and the most reliable way to confirm it on a four-pin module is to run an I2C scanner — the method used for this project.

2.2 Non-Blocking Sampling with millis()

In embedded systems, a microcontroller often needs to perform several tasks at once — reading sensor data, serving a web page, logging data to memory, responding to button presses, driving LEDs or motors. If the program stops and waits during one task, the others cannot run; this is called blocking.

The problem with delay()

delay() pauses the entire program for a fixed duration. While it waits, the ESP32 cannot handle HTTP requests, read buttons, blink LEDs, or perform any other task — which makes it unsuitable for a device that must also run a web server.

```
void loop() {  
  Serial.println("Reading Sensor...");  
  delay(10000); // CPU does nothing for 10 seconds  
}
```

The solution: millis()

Rather than stopping the program, `millis()` returns the number of milliseconds since the ESP32 started running, and the sketch compares elapsed time against a target interval to decide when to do recurring work — while everything else, including `server.handleClient()`, keeps running every pass of `loop()`.

```
unsigned long lastReadTime = 0;
const unsigned long interval = 10000; // 10 seconds

unsigned long currentTime = millis();
if (currentTime - lastReadTime >= interval) {
  lastReadTime = currentTime;
  // read the sensor
}
```

Why subtraction, not addition

The comparison is always written as `currentTime - lastReadTime >= interval`, never as `currentTime >= lastReadTime + interval`. Subtraction correctly handles the rollover of `millis()`, which wraps back to zero approximately every 49 days of continuous ESP32 uptime — the addition form would misbehave at that rollover, while the subtraction form remains correct due to how unsigned integer arithmetic wraps.

Program flow

```
loop() -> get current time -> has interval passed?
      No -> continue looping
      Yes -> read BME280 -> print values -> update lastReadTime -> continue looping
```

Real-world scheduling in this project

Task	Interval
Read BME280 sensor	Every 10 seconds
Append data to CSV	Every 10 seconds
Respond to HTTP requests	Continuously
Serve dashboard updates	Whenever requested by the browser

Using `millis()` lets all of these tasks coexist without pausing the program — the ESP32 stays responsive, multiple tasks run concurrently, and the pattern easily scales to more periodic tasks, which is why it is the preferred timing method for embedded and IoT applications.

2.3 Persistent Storage on ESP32: Flash, NVS, EEPROM Emulation, and SPIFFS/LittleFS

The ESP32 has exactly one physical non-volatile memory chip — Flash — which keeps its contents after power is removed. Everything that needs to survive a reboot (the program itself, configuration data, files, Wi-Fi credentials) ultimately lives inside that single chip; NVS, Preferences, EEPROM emulation, SPIFFS, and LittleFS are all software layers that manage different regions of it, not separate hardware.

```
Flash Memory
|-- Bootloader
|-- Partition Table
|-- Application (your program)
|-- OTA Partition
|-- NVS
|-- SPIFFS / LittleFS
`-- Other Partitions
```

Flash vs. NVS: not the same thing

Flash is the physical storage hardware; NVS (Non-Volatile Storage) is Espressif's built-in key-value database that manages a portion of that hardware. The relationship mirrors an SSD versus a file system: the SSD is hardware, the file system is software that organizes it, and your files are the data on top.

```
Preferences prefs;
prefs.begin("settings", false);
prefs.putInt("speed", 100);
prefs.putBool("enabled", true);
prefs.end();

// Internally stored as:
// Namespace: settings
//   speed   -> 100
//   enabled -> true
```

Why Preferences (NVS) replaces EEPROM

Espressif's official guidance is that EEPROM is deprecated on ESP32 and new applications should use Preferences instead — deprecated does not mean broken, it means the library still works for compatibility but is no longer recommended, because Preferences uses NVS directly and is faster, simpler, more flexible, and better optimized for Flash.

The ESP32 has no dedicated EEPROM hardware at all (unlike boards such as the Arduino Uno, which has Flash, RAM, and a separate EEPROM chip). The EEPROM library on ESP32 is purely an emulation layer: it creates one large binary block — a blob, short for Binary Large Object — inside NVS, and treats it as one big array of bytes.

```
EEPROM.write(5, 100); // modifies byte 5 inside the in-RAM copy of the blob
EEPROM.commit();      // writes the whole updated blob back into NVS
```

Because the whole blob is rewritten as a unit even if only one byte changed, EEPROM emulation is less efficient than Preferences, which stores each value independently — updating one key (e.g. count) never requires touching the others (e.g. speed, offset).

	EEPROM (emulation)	Preferences (NVS)
Storage model	Single binary blob, addressed by byte offset	Key-value pairs, addressed by name
Example call	EEPROM.write(0, 100); EEPROM.commit();	prefs.putInt("speed", 100);
Adding a new value	May require redesigning the whole memory layout	Just add a new key — nothing else changes

SPIFFS and LittleFS: real files, not variables

SPIFFS (SPI Flash File System) is a lightweight file system stored inside Flash that, unlike Preferences, stores files rather than individual variables — for example `/config.json`, `/log.txt`, `/index.html`, or `/data.csv`. LittleFS is the modern replacement for SPIFFS, offering better reliability, better long-term maintenance, improved power-loss recovery, and support for directories; unless SPIFFS is specifically required for compatibility, LittleFS is the recommended choice for new projects.

Preferences and SPIFFS/LittleFS typically report very different capacities (Preferences $\approx$ 20 KB, SPIFFS $\approx$ 1.5 MB on a common 4 MB Flash layout) not because they use different hardware, but because the partition table simply reserves different amounts of the same Flash chip for each region:

Region (4 MB Flash example)	Typical size
Application	1.5 MB
OTA	1 MB
NVS	20 KB
SPIFFS	1.5 MB

Storing a growing sensor log as individual Preferences keys (`temperature1`, `temperature2`, `temperature3`, ...) quickly becomes unmanageable; storing it as a file (`log.txt`, with one line appended per reading) is far easier to manage — which is the core reason this project logs to a CSV file on SPIFFS/LittleFS rather than to Preferences.

Performance characteristics

- Preferences — optimized for small variables, configuration values, counters, and calibration data; very fast for key-value operations.
- SPIFFS/LittleFS — optimized for reading, writing, and managing files and larger data; the better fit for file-based storage such as this project's CSV log.
- EEPROM (emulation) — designed to mimic traditional Arduino EEPROM behavior; works for porting older projects but is less flexible than Preferences for new work.

Wear leveling

Flash memory tolerates only a limited number of erase/write cycles per sector before it wears out. A good storage system spreads writes across multiple sectors — rotating between them — instead of repeatedly hitting the same one, extending the usable lifetime of the chip. Both NVS (and therefore Preferences) and LittleFS/SPIFFS implement wear-leveling internally, and because the ESP32's EEPROM emulation is itself built on NVS, it inherits the same protection.

Decision guidance

Full comparison

Feature	EEPROM (ESP32)	Preferences (NVS)	SPIFFS / LittleFS
Hardware	Flash	Flash	Flash
Storage type	Single binary blob	Key-value database	File system
Stores variables	Yes	Yes	No
Stores files	No	No	Yes
Uses addresses	Yes	No (uses keys)	No (uses paths)
Recommended for new projects	No (legacy)	Yes (settings)	Yes (files/logs)

2.4 ESP32 Wi-Fi Modes — STA, AP, and AP+STA

The ESP32's built-in Wi-Fi radio supports three operating modes, selected with `WiFi.mode()`.

1. Station mode (STA)

The ESP32 behaves like a normal Wi-Fi client — similar to a laptop or phone — joining an existing router using a known SSID and password.

```
WiFi.mode(WIFI_STA);  
WiFi.begin(ssid, password);
```

2. Access Point mode (AP) — used in this project

The ESP32 creates its own wireless network; other devices connect directly to it instead of to a router. This is the mode configured for the environmental monitor so it can be reached anywhere without existing Wi-Fi credentials.

```
WiFi.mode(WIFI_AP);
```

3. AP + STA mode

The ESP32 operates as both a Station connected to a router and an Access Point serving nearby devices simultaneously — useful for gateway-style projects, though not required for this task.

```
WiFi.mode(WIFI_AP_STA);
```

Important Wi-Fi terms

STA-mode connections can be established with a simple blocking wait for `WiFi.status() == WL_CONNECTED`, or more robustly with `WiFi.onEvent()` callbacks that fire automatically on connection, disconnection, or IP

assignment — an event-driven approach that keeps the CPU free for other work such as sensor reads. This project uses AP mode, where the ESP32 does not need to wait for a connection at all since it is itself the network.

2.5 Configuring and Using Access Point (AP) Mode

An Access Point is a device that creates a Wi-Fi network so other devices can join it. When the ESP32 operates as an Access Point it does not connect to an existing router — it becomes the network itself, which is useful for standalone embedded systems, initial device configuration portals, and demonstrations that must not depend on local Wi-Fi being available.

Configuration steps

```
#include <WiFi.h>
```

```
const char* AP_SSID = "ESP32_Monitor";  
const char* AP_PASSWORD = "12345678";
```

Built-in DHCP server

When operating as an Access Point, the ESP32 automatically runs a DHCP (Dynamic Host Configuration Protocol) server that assigns IP addresses to connecting clients — for example a laptop at 192.168.4.2 and a phone at 192.168.4.3 — entirely automatically, with no additional programming required.

AP mode vs. STA mode

Feature	AP Mode	STA Mode
Connects to a router	No	Yes
Creates its own Wi-Fi network	Yes	No
Requires an existing network	No	Yes
Provides IP addresses to clients	Yes (DHCP server)	No
Typical IP address	192.168.4.1	Assigned by the router (e.g. 192.168.1.105)

Advantages and limitations

- Advantages: no router required, works anywhere, easy to demonstrate and test, simple setup, well suited to configuration portals and local dashboards like this project's.
- Limitations: connecting devices must disconnect from their normal Wi-Fi network to join the ESP32; internet access is generally unavailable unless additional routing is implemented; range depends on the ESP32's radio and the environment.

Applied workflow for this project

```
Power on -> Initialize BME280 -> Initialize SPIFFS -> Start Access Point  
-> Laptop connects to ESP32 Wi-Fi -> Open browser -> http://192.168.4.1  
-> View live sensor dashboard -> Download CSV log
```

The result is a portable environmental monitoring system that can be accessed directly from any Wi-Fi-enabled device without requiring an external router or internet connection.

2.6 HTTP Request Routing with `server.on()`

`server.on()` binds a URL path to a callback function; when a client's HTTP request matches a registered path, the ESP32 runs that handler. This project registers three routes.

Route 1 — serving the dashboard ("/")

```
server.on("/", []() {  
  if (SPIFFS.exists("/index.html")) {  
    File file = SPIFFS.open("/index.html", "r");  
    server.streamFile(file, "text/html");  
    file.close();  
  } else {  
    server.send(404, "text/plain", "index.html not found in SPIFFS!");  
  }  
});
```

Route 2 — live JSON data ("/data")

```
server.on("/data", []() {  
  float temperature = bme.readTemperature();  
  float humidity = bme.readHumidity();  
  float pressure = bme.readPressure() / 100.0F;  
  
  String json = "{";  
  json += "\"temperature\": " + String(temperature, 2) + ",";  
  json += "\"humidity\": " + String(humidity, 2) + ",";  
  json += "\"pressure\": " + String(pressure, 2);  
  json += "}";  
  
  server.send(200, "application/json", json);  
});
```

```
{ "temperature": 24.50, "humidity": 48.20, "pressure": 1013.25 }
```

Route 3 — CSV download ("/download")

```
server.on("/download", []() {  
  if (SPIFFS.exists("/data.csv")) {  
    File file = SPIFFS.open("/data.csv", "r");  
    server.setHeader("Content-Type", "text/csv");
```

```
server.sendHeader("Content-Disposition", "attachment; filename=sensor_data.csv");
server.streamFile(file, "text/csv");
file.close();
} else {
server.send(404, "text/plain", "File not found");
}
});
```

Execution flow summary

```
setup(): SPIFFS.begin() -> server.on() registers /, /data, /download -> server.begin()
loop(): server.handleClient() runs repeatedly, invoking the matching
server.on() handler for each incoming request
```

3. References

- [BME280 Sensor with Arduino \(Pressure, Temperature, Humidity\) — Random Nerd Tutorials](#)
- [Circuit Designer — BME/BMP280 Component Reference](#)
- [ESP32 SPIFFS Storage Overview — TutorialsPoint](#)
- [SPIFFS on the ESP32 — Programming Electronics Academy](#)
- [ESP32 Essential Guide — Network Section \(AvantMaker\)](#)
- [Arduino-ESP32 Network API — Espressif Documentation](#)
- [Connect ESP32 to a Wi-Fi Network — DeepBlueMbedded](#)
- [ESP32 Access Point \(AP\) for Web Server — Random Nerd Tutorials](#)
- [ESP32 SPIFFS Tutorial: Storing Files & Building a Web Server — TeachMeMicro](#)

End of Report